CGE-P LAB CURRICULUM · V2

Lab 2.3: Building Your First Compliant Resource (AWS S3)

You've spent enough time in spreadsheets pretending they're controls. This lab ends that. You'll write Terraform for an S3 primitive that satisfies eleven NIST 800-53 controls, and produce machine-readable evidence of every one. No screenshots.

Every control in the table below is enforced by a line of code you will write. If you cannot point at the line, it does not go in the table. That discipline is the whole subject of this course.

For reading. Code lines wrap to fit the page; copy code from the repository, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from the repository (`GRCEngClub/cgep-labs`), not from the PDF.

Learning objectives

- Express NIST 800-53 controls as Terraform resources, citing each control at the line that enforces it.
- Enforce encryption rather than defaulting to it, in transit as well as at rest.
- Distinguish a control you implemented from a control you merely claimed.
- Build the primitive that the rest of the CGE-P labs reuse and verify automatically.

Controls implemented

Every row here is enforced by a line of code you will write. If you cannot point at the line, it does not go in the table.

Control	Enforced by	Note
SC-8, SC-8(1)	Bucket policy denying `aws:SecureTransport = false`	Transit encryption, the companion to SC-28.
SC-12, SC-13	`aws_kms_key` with `enable_key_rotation`	Key custody and rotation are yours.
SC-28, SC-28(1)	SSE-KMS default + policy denying wrong-key uploads	Enforced, not defaulted.
AC-3	Public access block, all four flags	
AC-6	`BucketOwnerEnforced` ownership	ACLs disabled.
AU-3	`aws_s3_bucket_logging` + delivery grant	Content of audit records.
AU-9	Versioning on the **log** bucket	Log objects cannot be silently overwritten.
AU-11	Lifecycle rules	Retention is a control and a budget.
CM-6, CM-8	Provider `default_tags`	Boundary enumeration by API.
CP-9, SI-7	Versioning on the primary bucket	

AU-6 is deliberately absent. AU-6 is audit *review, analysis, and reporting*. Shipping logs into a bucket is AU-3 plus AU-11; nothing in this lab reads them. Lab 5.2 earns AU-6 by standing up Athena over the trail and running a query. Noticing that a control has been collected rather than reviewed is a more valuable skill than closing the gap.

Prerequisites

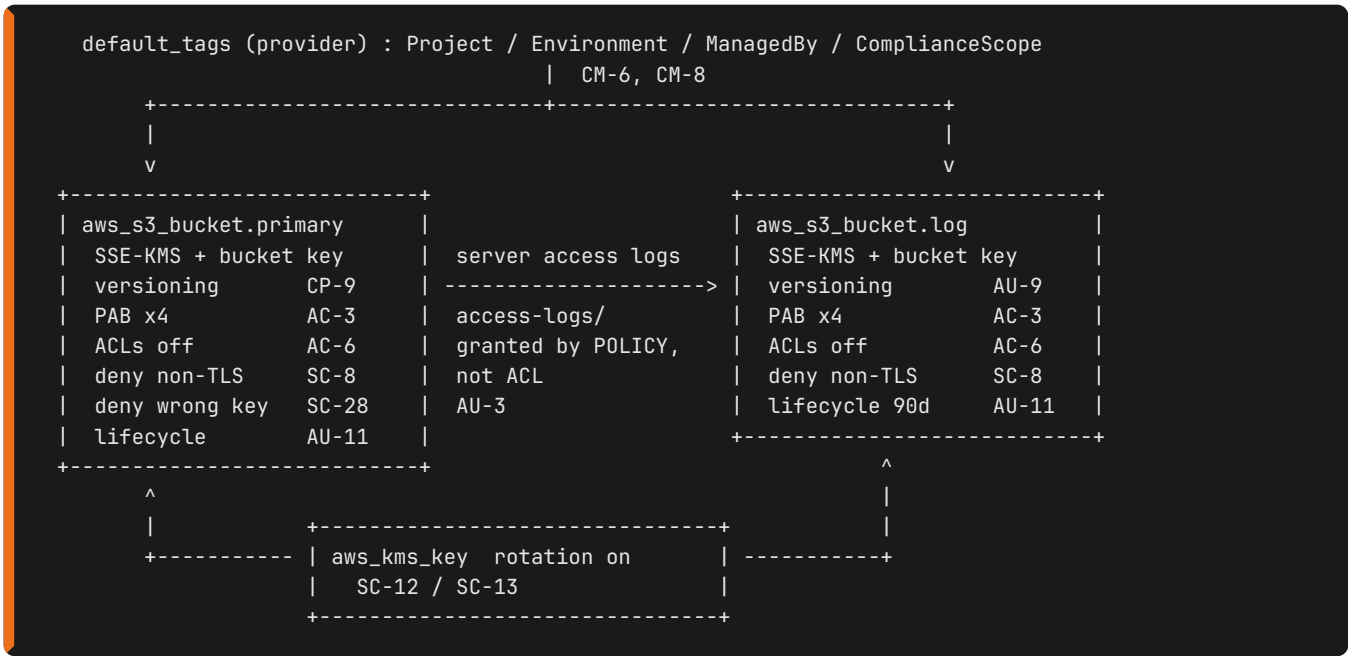
- **Lab 2.2 completed.** You need the state bucket name and the state CMK ARN.
- AWS account with permissions for S3, KMS, and IAM policy reads in `us-east-1`.
- Terraform `>= 1.10`.
- AWS CLI v2, with the `credential_process` profile from Lab 0.1 and `export AWS_PROFILE=cgep` set in this shell.

Estimated time & cost

- Time: 60 to 75 minutes the first time, 15 on repeats. The KMS key policy deserves reading rather than pasting, and that is most of the difference.
- Cost: **$1/month for the KMS key**, prorated to fractions of a cent if you destroy same-day, and it continues billing through the deletion waiting period. Empty S3 buckets have no idle cost. Under $0.02 total for a same-day run.

That KMS dollar is what buys SC-12 and SC-13. An AWS-managed key costs nothing and cannot satisfy either, because you can neither set its rotation policy nor read its key policy.

Architecture



Two things in that diagram are worth noticing now. The log bucket is versioned exactly like the primary, and log delivery is granted by a bucket policy rather than an ACL.

Step-by-step walkthrough

Step 1 Create the project structure

```
mkdir -p terraform/primitives/compliant-s3 && cd terraform/primitives/compliant-s3
touch main.tf variables.tf outputs.tf README.md
```

Step 2 Terraform block, backend, provider

```
# main.tf
terraform {
  required_version = ">= 1.10"

  # A PARTIAL backend configuration. The account-specific values are
  # supplied at init time from a file you do not commit; see below.
  backend "s3" {
    key          = "labs/2-3/terraform.tfstate"
    region       = "us-east-1"
    encrypt      = true
    use_lockfile = true
  }

  required_providers {
    aws    = { source = "hashicorp/aws", version = "~> 5.0" }
    random = { source = "hashicorp/random", version = "~> 3.6" }
  }
}

provider "aws" {
  region = var.aws_region

  # CM-6 / CM-8: required compliance tags on every taggable resource.
  # CM-6 is the configuration-settings argument; CM-8 is the stronger one,
  # because ComplianceScope is how you enumerate an authorization boundary
  # with an API call instead of a spreadsheet.
  default_tags {
    tags = {
      Project         = var.project_name
      Environment     = var.environment
      ManagedBy       = "terraform"
      ComplianceScope = "cge-p-lab"
    }
  }
}

data "aws_caller_identity" "current" {}
data "aws_partition" "current" {}

resource "random_id" "bucket_suffix" {
  byte_length = 4
```

```
}

locals {
  account_id = data.aws_caller_identity.current.account_id
  partition  = data.aws_partition.current.partition

  effective_suffix = coalesce(var.bucket_suffix, random_id.bucket_suffix.hex)
  primary_name     = "${var.project_name}-${var.environment}-data-${local.effective_suffix}"
  log_name         = "${var.project_name}-${var.environment}-logs-${local.effective_suffix}"
}
```

`coalesce` skips both `null` and the empty string, which is why the variable defaults to `null` and the intent reads directly. The longhand alternative is a `var.bucket_suffix != "" ? ... : ...` ternary.

The bucket-name arithmetic is not arbitrary, and it constrains the validation you write next. Worst case: `project_name` at 21, plus `-staging-data-`, plus 8 hex characters, is 43 characters. S3's limit is 63. **The regex ceiling in `variables.tf` was reverse-engineered from this naming scheme.** Change the scheme, move the regex.

Why the backend block is incomplete. Your bucket name and key ARN both carry your AWS account ID, and the capstone requires your repository to be **public**. Hardcoding them here would teach you to commit account identifiers, which is the opposite of the habit this course is building.

Terraform supports partial backend configuration: leave the account-specific keys out of the block and supply them at init. Build `backend.hcl` from Lab 2.2's outputs rather than transcribing them:

```
cd ../lab-2-2
printf 'bucket     = "%s"\nkms_key_id = "%s"\n' \
  "$(terraform output -raw state_bucket)" \
  "$(terraform output -raw state_kms_key_arn)" > ../lab-2-3/backend.hcl
```

Add `backend.hcl` to `.gitignore`, then initialize with it:

```
terraform init -backend-config=backend.hcl
```

Omit `-backend-config` and Terraform prompts for the two missing values, so the workspace still runs for someone who has not made the file. Commit a `backend.hcl.example` holding placeholders so the next person knows what is expected. Every later lab uses the same pattern with its own `key`.

Step 3 variables.tf

```
# variables.tf
variable "project_name" {
  type        = string
  description = "Short project identifier. Becomes part of bucket names and the Project tag."
  validation {
    condition     = can(regex("^[a-z][a-z0-9-]{2,20}$", var.project_name))
    error_message = "project_name must be 3-21 lowercase alphanumerics or hyphens, starting with a
letter."
  }
}

variable "environment" {
  type        = string
  description = "Deployment environment. Drives the Environment tag and downstream policy decisions."
  validation {
    condition     = contains(["dev", "staging", "prod"], var.environment)
    error_message = "environment must be one of: dev, staging, prod."
  }
}

variable "aws_region" {
  type        = string
  description = "Region. Must match the region of the Lab 2.2 state backend."
  default     = "us-east-1"
}

variable "bucket_suffix" {
  type        = string
  description = "Optional suffix to force a specific bucket name. Defaults to a random_id."
  default     = null
}

variable "kms_deletion_window_days" {
  type        = number
  description = "Waiting period before a scheduled CMK deletion completes. AWS allows 7-30."
  default     = 7
  validation {
    condition     = var.kms_deletion_window_days >= 7 && var.kms_deletion_window_days <= 30
    error_message = "kms_deletion_window_days must be between 7 and 30."
  }
}

variable "log_retention_days" {
  type        = number
  description = "AU-11. How long access-log objects are kept before expiry."
  default     = 90
  validation {
    condition     = var.log_retention_days >= 1
    error_message = "log_retention_days must be at least 1."
  }
}

variable "noncurrent_version_retention_days" {
```

```
  type        = number
  description = "How long superseded object versions survive before expiry."
  default     = 30
}
```

Validation blocks are **preventive** controls. They reject bad input at plan time, before anything reaches AWS. Compare that to detecting a mistyped `Environment` tag in a Config rule three hours after deploy, which is **detective** and strictly weaker. That distinction is worth more to you than the regex.

Step 4 The KMS key

```
# main.tf (continued)

# ------------------------------------------------------------------------------
# SC-12 (key establishment and management) + SC-13 (cryptographic protection)
# A customer-managed key with rotation. This is the control an AWS-managed key
# cannot satisfy: you cannot set rotation policy or read the key policy of one.
# ------------------------------------------------------------------------------
resource "aws_kms_key" "bucket" {
  description             = "CMK for ${local.primary_name} and its access-log bucket"
  enable_key_rotation     = true
  deletion_window_in_days = var.kms_deletion_window_days
  policy                  = data.aws_iam_policy_document.kms.json
}

resource "aws_kms_alias" "bucket" {
  name          = "alias/${var.project_name}-${var.environment}-s3"
  target_key_id = aws_kms_key.bucket.key_id
}

data "aws_iam_policy_document" "kms" {
  # Omitting this statement makes the key permanently unmanageable. AWS will
  # let you do it. Do not.
  statement {
    sid       = "EnableAccountRootAdmin"
    effect    = "Allow"
    actions   = ["kms:*"]
    resources = ["*"]
    principals {
      type        = "AWS"
      identifiers = ["arn:${local.partition}:iam::${local.account_id}:root"]
    }
  }

  # S3 server access log delivery encrypts each log object with this key, so
  # the logging service principal needs to use it. The SourceAccount condition
  # is confused-deputy protection: it stops another account from inducing the
  # S3 service to use your key on their behalf.
  statement {
    sid       = "AllowS3LogDelivery"
    effect    = "Allow"
    actions   = ["kms:GenerateDataKey", "kms:Decrypt"]
```

```
    resources = ["*"]
    principals {
      type        = "Service"
      identifiers = ["logging.s3.amazonaws.com"]
    }
    condition {
      test     = "StringEquals"
      variable = "aws:SourceAccount"
      values   = [local.account_id]
    }
  }
}
```

`resources = ["*"]` inside a *key policy* means "this key," not "every key." Key policies are attached to one key and scoped to it. This confuses everybody once.

> **Constraint worth knowing before you hit it.** An S3 server-access-log *destination* bucket encrypted with SSE-KMS requires **S3 Bucket Keys enabled** on that bucket. This is why `bucket_key_enabled = true` appears on the log bucket below and is not optional there. If log objects never appear, this is the first thing to check.

Step 5 The two buckets and the shared baseline

```
# main.tf (continued)

resource "aws_s3_bucket" "primary" {
  bucket = local.primary_name
}

resource "aws_s3_bucket" "log" {
  bucket = local.log_name
}

# AC-6: ACLs disabled entirely. AWS's default for new buckets since April 2023.
# The tempting alternative, BucketOwnerPreferred plus a log-delivery-write ACL,
# re-enables ACLs on the log bucket and trips Security Hub control S3.12. Log
# delivery is granted by bucket policy instead; see Step 7.
resource "aws_s3_bucket_ownership_controls" "primary" {
  bucket = aws_s3_bucket.primary.id
  rule {
    object_ownership = "BucketOwnerEnforced"
  }
}

resource "aws_s3_bucket_ownership_controls" "log" {
  bucket = aws_s3_bucket.log.id
  rule {
    object_ownership = "BucketOwnerEnforced"
  }
```

```
}

# SC-28: encryption at rest with the CMK, not the AWS-managed default.
resource "aws_s3_bucket_server_side_encryption_configuration" "primary" {
  bucket = aws_s3_bucket.primary.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm     = "aws:kms"
      kms_master_key_id = aws_kms_key.bucket.arn
    }
    bucket_key_enabled = true
  }
}

# bucket_key_enabled is REQUIRED here, not merely economical: an SSE-KMS
# server-access-log destination bucket must have S3 Bucket Keys on.
resource "aws_s3_bucket_server_side_encryption_configuration" "log" {
  bucket = aws_s3_bucket.log.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm     = "aws:kms"
      kms_master_key_id = aws_kms_key.bucket.arn
    }
    bucket_key_enabled = true
  }
}

# CP-9 / SI-7 on the primary: prior object states survive deletion and overwrite.
resource "aws_s3_bucket_versioning" "primary" {
  bucket = aws_s3_bucket.primary.id
  versioning_configuration {
    status = "Enabled"
  }
}

# AU-9 on the log bucket: protection of audit information.
# Easy to omit, and the omission is quiet. Without it, log objects can be
# overwritten in place, which leaves the bucket holding your audit records
# less protected than the bucket holding your data.
resource "aws_s3_bucket_versioning" "log" {
  bucket = aws_s3_bucket.log.id
  versioning_configuration {
    status = "Enabled"
  }
}

# AC-3: all four flags. Three is not enough.
resource "aws_s3_bucket_public_access_block" "primary" {
  bucket                  = aws_s3_bucket.primary.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_public_access_block" "log" {
```

```
  bucket                  = aws_s3_bucket.log.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

Those four flags are not four copies of one idea. They are a 2x2:

	Blocks *new*	**Neutralizes** *existing*
ACLs	`block_public_acls`	`ignore_public_acls`
Policies	`block_public_policy`	`restrict_public_buckets`

Set three and you have either left an existing public grant live or left the door open for a new one.

Step 6 Lifecycle, for AU-11 and for your bill

```
# main.tf (continued)

# AU-11: audit record retention. Also the difference between a lab and an
# unbounded S3 invoice.
resource "aws_s3_bucket_lifecycle_configuration" "log" {
  bucket     = aws_s3_bucket.log.id
  depends_on = [aws_s3_bucket_versioning.log]

  rule {
    id     = "expire-access-logs"
    status = "Enabled"

    # An empty filter applies the rule to all objects. Provider 5.x rejects a
    # rule that has neither filter nor prefix.
    filter {}

    expiration {
      days = var.log_retention_days
    }

    noncurrent_version_expiration {
      noncurrent_days = var.noncurrent_version_retention_days
    }

    abort_incomplete_multipart_upload {
      days_after_initiation = 7
    }
  }
}

resource "aws_s3_bucket_lifecycle_configuration" "primary" {
  bucket     = aws_s3_bucket.primary.id
```

```
  depends_on = [aws_s3_bucket_versioning.primary]

  rule {
    id     = "expire-noncurrent-versions"
    status = "Enabled"
    filter {}

    noncurrent_version_expiration {
      noncurrent_days = var.noncurrent_version_retention_days
    }

    abort_incomplete_multipart_upload {
      days_after_initiation = 7
    }
  }
}
```

Note what the primary bucket's rule does *not* do: it never expires current object versions. Expiring live data is a business decision, not a compliance default. Superseded versions and abandoned multipart uploads are pure carrying cost, so those go.

Step 7 Bucket policies

```
# main.tf (continued)

data "aws_iam_policy_document" "primary" {
  # SC-8: transmission confidentiality. Refuse anything not over TLS.
  statement {
    sid       = "DenyInsecureTransport"
    effect    = "Deny"
    actions   = ["s3:*"]
    resources = [aws_s3_bucket.primary.arn, "${aws_s3_bucket.primary.arn}/*"]
    principals {
      type        = "*"
      identifiers = ["*"]
    }
    condition {
      test     = "Bool"
      variable = "aws:SecureTransport"
      values   = ["false"]
    }
  }

  # SC-28 enforcement: encryption becomes a condition of upload, not a default
  # applied on your behalf. Read the warning below before you apply this.
  statement {
    sid       = "DenyUnencryptedObjectUploads"
    effect    = "Deny"
    actions   = ["s3:PutObject"]
    resources = ["${aws_s3_bucket.primary.arn}/*"]
    principals {
      type        = "*"
```

```
      identifiers = ["*"]
    }
    condition {
      test     = "StringNotEquals"
      variable = "s3:x-amz-server-side-encryption"
      values   = ["aws:kms"]
    }
  }

  # SC-28(1): the right algorithm with the wrong key is not the control.
  statement {
    sid       = "DenyWrongKmsKey"
    effect    = "Deny"
    actions   = ["s3:PutObject"]
    resources = ["${aws_s3_bucket.primary.arn}/*"]
    principals {
      type        = "*"
      identifiers = ["*"]
    }
    condition {
      test     = "StringNotEquals"
      variable = "s3:x-amz-server-side-encryption-aws-kms-key-id"
      values   = [aws_kms_key.bucket.arn]
    }
  }
}

resource "aws_s3_bucket_policy" "primary" {
  bucket = aws_s3_bucket.primary.id
  policy = data.aws_iam_policy_document.primary.json
}

data "aws_iam_policy_document" "log" {
  statement {
    sid       = "DenyInsecureTransport"
    effect    = "Deny"
    actions   = ["s3:*"]
    resources = [aws_s3_bucket.log.arn, "${aws_s3_bucket.log.arn}/*"]
    principals {
      type        = "*"
      identifiers = ["*"]
    }
    condition {
      test     = "Bool"
      variable = "aws:SecureTransport"
      values   = ["false"]
    }
  }

  # AU-3: the modern replacement for the log-delivery-write ACL.
  # Both conditions matter. SourceArn scopes the grant to exactly one source
  # bucket; SourceAccount stops a different account from using the S3 logging
  # service as a confused deputy to write into your bucket.
  statement {
    sid       = "AllowS3ServerAccessLogDelivery"
    effect    = "Allow"
```

```
    actions   = ["s3:PutObject"]
    resources = ["${aws_s3_bucket.log.arn}/access-logs/*"]
    principals {
      type        = "Service"
      identifiers = ["logging.s3.amazonaws.com"]
    }
    condition {
      test     = "ArnLike"
      variable = "aws:SourceArn"
      values   = [aws_s3_bucket.primary.arn]
    }
    condition {
      test     = "StringEquals"
      variable = "aws:SourceAccount"
      values   = [local.account_id]
    }
  }
}

resource "aws_s3_bucket_policy" "log" {
  bucket = aws_s3_bucket.log.id
  policy = data.aws_iam_policy_document.log.json
}

# AU-3: wire the primary bucket's access logs into the log bucket.
# depends_on is genuinely required: the delivery grant must exist before S3
# will accept the logging configuration, and nothing here references the
# policy, so Terraform cannot infer the order. Most depends_on in the wild is
# cargo cult; this one is not.
resource "aws_s3_bucket_logging" "primary" {
  bucket        = aws_s3_bucket.primary.id
  target_bucket = aws_s3_bucket.log.id
  target_prefix = "access-logs/"

  depends_on = [aws_s3_bucket_policy.log]
}
```

Read this before you apply. `DenyUnencryptedObjectUploads` uses `StringNotEquals`, and in IAM a `StringNotEquals` condition evaluates to **true when the key is absent**. So this Deny fires on any `PutObject` that does not explicitly carry the `x-amz-server-side-encryption` header, including uploads that the bucket's own default encryption would have encrypted anyway.

That is the intended behavior and it has a real usability cost. Uploads must be explicit:

```
aws s3 cp ./file.txt "s3://$BUCKET/file.txt" \
  --sse aws:kms --sse-kms-key-id "$KMS_ARN" --profile <your-sandbox>
```

Deciding whether to accept that friction is a genuine control-design trade-off, and defending your answer is exactly the kind of reasoning the capstone write-up scores. If you decide against it, delete the statement and say why in your README. Do not leave it in and route around it.

Note what is deliberately **not** on the log bucket: the SSE-enforcement denies. The S3 log delivery service does not send an `x-amz-server-side-encryption` header, so adding those statements there silently breaks log delivery. The bucket's default encryption still applies. This asymmetry is the single easiest way to break this lab.

Step 8 outputs.tf

```
# outputs.tf
output "bucket_name" {
  value       = aws_s3_bucket.primary.bucket
  description = "Primary bucket name."
}

output "bucket_arn" {
  value       = aws_s3_bucket.primary.arn
  description = "Primary bucket ARN."
}

output "log_bucket_name" {
  value       = aws_s3_bucket.log.bucket
  description = "Access-log bucket name."
}

output "log_bucket_arn" {
  value       = aws_s3_bucket.log.arn
  description = "Access-log bucket ARN."
}

output "kms_key_arn" {
  value       = aws_kms_key.bucket.arn
  description = "CMK protecting both buckets (SC-12 / SC-13 attestation)."
}

# The attestation is the module's evidence surface. Lab 2.4 builds the same
# thing on GCP; Lab 6.1's OSCAL component points at the JSON this lands in.
output "compliance_attestation" {
  description = "Computed attestation of the controls this primitive enforces."
  value = {
    encryption_algorithm = one([
      for rule in aws_s3_bucket_server_side_encryption_configuration.primary.rule :
      rule.apply_server_side_encryption_by_default[0].sse_algorithm
    ])
    kms_key_arn             = aws_kms_key.bucket.arn
    kms_rotation_enabled    = aws_kms_key.bucket.enable_key_rotation
    primary_versioning      = aws_s3_bucket_versioning.primary.versioning_configuration[0].status
    log_versioning          = aws_s3_bucket_versioning.log.versioning_configuration[0].status
    public_access_blocked   = alltrue([
      aws_s3_bucket_public_access_block.primary.block_public_acls,
      aws_s3_bucket_public_access_block.primary.block_public_policy,
      aws_s3_bucket_public_access_block.primary.ignore_public_acls,
      aws_s3_bucket_public_access_block.primary.restrict_public_buckets,
    ])
    acls_disabled = one([
```

```
      for rule in aws_s3_bucket_ownership_controls.primary.rule : rule.object_ownership
    ]) == "BucketOwnerEnforced"
    tls_required          = true
    access_logging_target = aws_s3_bucket_logging.primary.target_bucket
    log_retention_days    = var.log_retention_days
  }
}
```

Why the `one(...)` expression. Terraform models the `rule` block of `aws_s3_bucket_server_side_encryption_configuration` as a **set**, and sets are not index-addressable. The `for` expression flattens it to a tuple, then `one()` extracts the single element.

The alternative, `tolist(...)[0]`, works and is worse. If there were ever two rules, it returns an arbitrary one silently; `one()` raises an error. **This output is an attestation, so failing loudly beats attesting confidently to the wrong value.** That instinct is the difference between an evidence pipeline and a liability.

Step 9 init / plan / apply

`project_name` and `environment` are the two variables with no default, which is deliberate: they name your buckets and drive the `Environment` tag, and a default would let you deploy to the wrong environment by forgetting a flag. Give them values in a `terraform.tfvars`, which Terraform reads automatically:

```
cat > terraform.tfvars <<'EOF'
project_name = "cgep-lab"
environment  = "dev"
EOF
```

`*.tfvars` is gitignored, so this stays on your machine. Skip this step and Terraform silently prompts for both values on every `plan`, `apply` and `destroy`, and fails outright under `-input=false`, which is how CI runs.

If you set up `cgep.env` in Lab 0.1 Step 15, you already have these as `TF_VAR_project_name` and `TF_VAR_environment`, and you can skip the file entirely. `source cgep.env` and Terraform finds them on its own.

```
export AWS_PROFILE=cgep
terraform init
terraform fmt
terraform validate
terraform plan -out=tfplan
terraform apply -auto-approve tfplan
```

Run `terraform fmt` from **this directory**. It rewrites in place and prints only the names of files it changed, so silence means either "already formatted" or "no `.tf` files here." Use `terraform fmt -check` and read the exit code if you want certainty: `0` formatted, `3` changes needed, `1` error.

Expected tail:

```
Apply complete! Resources: 18 added, 0 changed, 0 destroyed.

Outputs:

bucket_arn = "arn:aws:s3:::cgep-lab-dev-data-XXXXXXXX"
bucket_name = "cgep-lab-dev-data-XXXXXXXX"
compliance_attestation = {
  "access_logging_target" = "cgep-lab-dev-logs-XXXXXXXX"
  "acls_disabled" = true
  "encryption_algorithm" = "aws:kms"
  "kms_key_arn" = "arn:aws:kms:us-east-1:...:key/..."
  "kms_rotation_enabled" = true
  "log_retention_days" = 90
  "log_versioning" = "Enabled"
  "primary_versioning" = "Enabled"
  "public_access_blocked" = true
  "tls_required" = true
}
kms_key_arn = "arn:aws:kms:us-east-1:...:key/..."
log_bucket_arn = "arn:aws:s3:::cgep-lab-dev-logs-XXXXXXXX"
log_bucket_name = "cgep-lab-dev-logs-XXXXXXXX"
```

Eighteen resources: the random suffix, the KMS key and its alias, the two buckets, and the configuration resources that harden each of them.

Step 10 Capture evidence

```
mkdir -p evidence/lab-2-3
terraform show -json tfplan > evidence/lab-2-3/plan.json
terraform show -json        > evidence/lab-2-3/state.json
terraform output -json compliance_attestation > evidence/lab-2-3/attestation.json
```

Open `state.json` and find each control rather than taking the table's word for it:

- SC-28 at `server_side_encryption_configuration[].rule[].apply_server_side_encryption_by_default[].sse_algorithm`
- SC-8 in the primary bucket policy's `DenyInsecureTransport` statement
- AC-3 in the four `true`s
- AU-9 in `aws_s3_bucket_versioning.log`
- CM-6 in `tags_all`

`terraform show -json` output **is** machine-readable compliance evidence. No screenshots.

> **One caution the pipeline chapters depend on.** `state.json` for other workspaces can contain secrets in plaintext. This workspace's state does not, but the `capture-evidence.sh` script in Lab 2.5 runs `terraform state pull` and uploads the result into an Object Lock vault. Uploading a secret into immutable storage means you cannot delete it. Lab 2.5 addresses this directly.

Verification

```
BUCKET=$(terraform output -raw bucket_name)
LOGS=$(terraform output -raw log_bucket_name)
KMS=$(terraform output -raw kms_key_arn)

# SC-28: KMS, not AES256
aws s3api get-bucket-encryption --bucket "$BUCKET"

# SC-12/SC-13: rotation on
aws kms get-key-rotation-status --key-id "$KMS"

# CP-9 and AU-9: versioning on BOTH buckets
aws s3api get-bucket-versioning --bucket "$BUCKET"
aws s3api get-bucket-versioning --bucket "$LOGS"

# AC-3
aws s3api get-public-access-block --bucket "$BUCKET"

# AC-6: ACLs disabled
aws s3api get-bucket-ownership-controls --bucket "$BUCKET"

# AU-11
aws s3api get-bucket-lifecycle-configuration --bucket "$LOGS"

# AU-3
aws s3api get-bucket-logging --bucket "$BUCKET"
```

Then make the controls actually deny something. A control you have not watched fire is a control you are guessing about:

```
# SC-8: plain HTTP must be refused
aws s3api list-objects-v2 --bucket "$BUCKET" \
  --endpoint-url "http://s3.us-east-1.amazonaws.com"
# expect: AccessDenied

# SC-28: an upload with no encryption header must be refused
echo test > /tmp/t.txt
aws s3 cp /tmp/t.txt "s3://$BUCKET/t.txt"
# expect: AccessDenied

# ...and the same upload, done correctly, must succeed
aws s3 cp /tmp/t.txt "s3://$BUCKET/t.txt" \
  --sse aws:kms --sse-kms-key-id "$KMS"
# expect: upload succeeds
```

Three denials and one success. That sequence is better evidence than any of the `get-*` calls, because it demonstrates enforcement rather than configuration.

Portfolio submission checklist

- [ ] `terraform/primitives/compliant-s3/{main.tf,variables.tf,outputs.tf,README.md}`
- [ ] `README.md`, one paragraph naming the eleven controls and stating plainly that AU-6 is **not** among them and why.
- [ ] `evidence/lab-2-3/plan.json`
- [ ] `evidence/lab-2-3/state.json`
- [ ] `evidence/lab-2-3/attestation.json`
- [ ] `evidence/lab-2-3/enforcement-test.txt`, a transcript of the three denials and one success above.

That last item is new in v2 and it is the one an assessor would actually want.

Troubleshooting

- **`Credentials were refreshed, but the refreshed credentials are still expired.`** If it appears seconds after `aws login`, it is a transient cache race: the CLI returns your prompt before its own credential cache settles. Re-run the command. If it persists, the shell is holding an expired snapshot from `export-credentials --format env`, which outranks `AWS_PROFILE`: `unset AWS_ACCESS_KEY_ID AWS_SECRET_ACCESS_KEY AWS_SESSION_TOKEN AWS_CREDENTIAL_EXPIRATION`, then re-run.
- **`ExpiredToken` from Terraform mid-lab.** `aws login` sessions are short, on the order of fifteen minutes, and a lab is longer than that. Run `aws login --profile default` and re-run the Terraform command; with the `credential_process` profile there is nothing to re-export, because the provider resolves credentials afresh on every run. With remote state nothing is lost. Short-lived credentials expiring is the feature you chose by not creating an access key.
- **`BucketAlreadyExists`.** S3 names are globally unique across every AWS account. The `random_id` suffix prevents this unless you set `bucket_suffix` by hand.
- **Access log objects never appear.** Three causes, in order of likelihood: (1) `bucket_key_enabled` is not `true` on the SSE-KMS log bucket, which server access logging requires; (2) the KMS key policy is missing the `logging.s3.amazonaws.com` grant; (3) you have been waiting less than a few hours. Server access log delivery is best-effort and slow. Generate traffic against the primary bucket first, or there is nothing to log.
- **`AccessDenied` on every upload after apply.** Working as designed. See the `DenyUnencryptedObjectUploads` warning in Step 7 and pass `--sse aws:kms --sse-kms-key-id`.
- **`MalformedPolicy: Policy has invalid action`** on the log bucket. `logging.s3.amazonaws.com` needs `s3:PutObject` on the object path (`arn/access-logs/*`), not on the bucket ARN.
- **`InvalidArgument` on the lifecycle rule.** Provider 5.x requires each rule to have `filter` or `prefix`. An empty `filter {}` means "all objects."
- **`failed to find SSO session section`.** Terraform's AWS provider does not always parse SSO config the way the CLI does. The `credential_process` profile sidesteps it entirely, because resolution happens in the CLI rather than in the provider.

- **`Error acquiring the state lock`.** Leftover from an interrupted run. With Lab 2.2's `use_lockfile`, the lock is an object at `labs/2-3/terraform.tfstate.tflock`. Confirm nobody else is applying, then `terraform force-unlock <lock-id>`.
- **Region mismatch on verification.** The provider wins for resource creation; `aws s3api` reads fall back to your profile's region. Pass `--region us-east-1` if a check returns `NoSuchBucket`.

Cleanup

Versioned buckets refuse to be destroyed while they hold any object version, and now **both** buckets are versioned. Empty both, including delete markers:

```
for B in "$(terraform output -raw bucket_name)" "$(terraform output -raw log_bucket_name)"; do
  aws s3api list-object-versions --bucket "$B" --output json \
    | python3 -c 'import sys,json; d=json.load(sys.stdin); items=[*d.get("Versions",
[]),*d.get("DeleteMarkers",[])]; print(json.dumps({"Objects":
[{"Key":o["Key"],"VersionId":o["VersionId"]} for o in items]}))' > /tmp/del.json
  # delete-objects rejects an empty Objects list, so skip when there is nothing to do
  if [ "$(jq '.Objects | length' /tmp/del.json)" -gt 0 ]; then
    aws s3api delete-objects --bucket "$B" --delete file:///tmp/del.json
  fi
done

terraform destroy -auto-approve
```

The `jq` length guard matters. `delete-objects` rejects an empty `Objects` list, and the tempting fix is a trailing `|| true`, which swallows that error along with every real failure.

The KMS key does not disappear. It enters the deletion waiting period you set with `kms_deletion_window_days` (minimum 7) and **bills the full $1/month throughout**. That is unavoidable; AWS deliberately makes key destruction slow.

How this feeds the capstone

This is your first compliant primitive, and by the end of the course you will have a dozen.

- **Ch 3**, your Rego policies read this exact `plan.json`. Because v2 adds SC-8 and CMK enforcement, Lab 3.4's policy library gains rules for both, and the `compliance_attestation` output gives the policies a stable shape to assert against.
- **Ch 4**, the `plan + show -json` ritual becomes a CI step, and `evidence/lab-2-3/` seeds the signed bundles. Worth knowing now: Chapter 4's gate runs Trivy, which flags both a missing HTTPS-enforcement policy and non-CMK encryption at HIGH. Build the bucket the way this lab does and the gate stays quiet; take either shortcut and your own pipeline will tell you so two chapters later.
- **Ch 6**, you write an OSCAL Component Definition for this module. SC-28's `implemented-requirement` points at `evidence/lab-2-3/state.json`. An assessor reads the OSCAL, follows the link, sees the same JSON you generated. The audit becomes a traversal.

- **Capstone Layer 1**, the required "hardening overrides" on the starter's uploads bucket are this pattern applied to a bucket you did not create. The capstone also requires a customer-managed key with rotation, which is the key you build in Step 4.

Revision history

v2 (current)

- Encryption moved from SSE-S3 (`AES256`) to a customer-managed KMS key with rotation and bucket keys, adding SC-12 and SC-13.
- Added a bucket policy denying `aws:SecureTransport = false`, adding SC-8 and SC-8(1).
- Added policy statements denying uploads that do not name the correct key, making SC-28 enforced rather than defaulted.
- Added versioning to the access-log bucket, adding AU-9. Previously only the primary bucket was versioned.
- Added lifecycle configuration to both buckets, adding AU-11.
- Switched log delivery from a `log-delivery-write` ACL to a bucket policy grant with `aws:SourceArn` and `aws:SourceAccount` conditions, and set `BucketOwnerEnforced` on both buckets, adding AC-6.
- Dropped the AU-6 claim. Collecting logs is AU-3 and AU-11; AU-6 requires review, which Lab 5.2 now provides.
- Remapped versioning from CM-6 to CP-9 and SI-7, and added CM-8 alongside CM-6 for the tagging control.
- Added `compliance_attestation` output as a machine-readable evidence surface.
- Resource count 11 to 18.

v1

Initial release: five controls claimed (SC-28, AU-3, AU-6, CM-6, AC-3), SSE-S3 encryption, one versioned bucket, ACL-based log delivery.